AutoE2E Multi-Benchmark Evaluation Report

Date: 2026-03-12 (original PETCLINIC evaluation: 2026-03-10)
Repository: `autoe2e` (AutoE2E: Feature-Driven End-To-End Test Generation)
Benchmarks: Spring PetClinic Angular (`PETCLINIC`), Ever Traduora (`EVERTRADUORA`), Posts & Users (`POSTUSERS`), Heroes Angular (`HEROES`), Airbus Inventory Management System (`AIRBUS`), SoftScanner CQA (`SOFTSCANNER_CQA`)
Evaluator: Automated pipeline execution + code inspection

Executive Summary

AutoE2E presents itself as a system for "Feature-Driven End-To-End Test Generation." This report documents the results of executing the pipeline against six empirical benchmark applications across three categories: two from the repository's own `benchmark/` folder (PETCLINIC, EVERTRADUORA) and four external local applications (Posts & Users, Heroes Angular, Airbus Inventory Management System, SoftScanner CQA). All six runs involved full startup of the benchmark application, pipeline execution, and post-run artifact inspection.

Key findings:

1. **AutoE2E does not generate executable tests.** This conclusion is established by static code inspection and confirmed by six independent benchmark runs. No test files, assertions, or test-framework output are produced anywhere in the pipeline.
2. **What it actually produces** is a feature discovery and action-feature mapping system: a state graph (JSON), a feature database (MongoDB), and an action-to-feature mapping database (MongoDB).
3. **PETCLINIC run (2026-03-10):** 9 states, 9 transitions, 72 actions, 46 features, 9 screenshots. Covered homepage navigation, owner/vet management, pet types, specialties. Binding constraint: 30-action ceiling.
4. **EVERTRADUORA run (2026-03-12):** 3 states, 3 transitions, 13 actions, 5 features, 3 screenshots. Constrained by an authentication wall — only login, signup, and forgot-password were reachable. Binding constraint: authentication wall.
5. **POSTUSERS run (2026-03-12):** 7 states, 7 transitions, 95 total actions (30 executed), 37 raw features, 7 screenshots. Covered posts list, post creation form, user list, user registration form (38-field form), user search, availability scheduling. Binding constraint: 30-action ceiling. Empty backend prevented dynamic routes (`/users/:id`) from being reached.
6. **HEROES run (2026-03-12):** 5 states, 5 transitions, 53 total actions (30 executed), 58 raw LLM-inferred features (~25 unique after deduplication), 5 screenshots. Covered heroes list (with 6 seeded entries), villains list, about page, hero-add form, and hero-edit form — all as distinct DOM states within 2 unique URLs. Binding constraint: 30-action ceiling.
7. **AIRBUS run (2026-03-12):** 1 state, 1 transition (self-loop), 3 candidate actions, 6 action-processing iterations (3 actions processed on each of 2 revisits to the same state), 4 raw LLM-inferred features, **0 features stored in MongoDB**, 1 screenshot. Only the login page was reachable — all other routes (product dashboard, add/update/category views) are guarded by JWT authentication. The FormAction was detected but could not be submitted (no `data-testid` attributes). This is the most restricted run of the six. Binding constraint: authentication wall.
8. **Pre-seeded data qualitatively changes pipeline coverage.** HEROES (JSON Server with `db.json`) and PETCLINIC (Docker REST API with data) both show populated list pages, enabling the crawler to interact with individual records and reach inline edit/add form states. POSTUSERS (empty in-memory backend), EVERTRADUORA (auth-blocked), and AIRBUS (fully auth-blocked) could not reach record-level states.
9. **EVERTRADUORA, POSTUSERS, AIRBUS, and SOFTSCANNER_CQA confirmed that the pipeline's form submission subsystem is PetClinic-specific**, requiring `data-testid` , `data-formid` , and `data-submitid` attributes absent in standard web applications. HEROES uses Angular reactive forms without standard `<form>` elements — the candidate action extractor found no FormAction in those views, so the issue was entirely avoided. AIRBUS detected the login form as a FormAction but could not submit it. SOFTSCANNER_CQA uses Angular Material reactive forms with standard `<input>` elements and `id` attributes — no `data-testid` attributes are present, so no form submission occurred.
10. **The pipeline's config-level extensibility to external applications is confirmed across four external apps.** POSTUSERS, HEROES, AIRBUS, and SOFTSCANNER_CQA were all integrated via a single 12-line JSON config file. No AutoE2E source changes were needed beyond those already made for EVERTRADUORA. HEROES and AIRBUS required `NODE_OPTIONS=--openssl-legacy-provider` (Angular 11/12 + Node.js 22 OpenSSL incompatibility); SOFTSCANNER_CQA (Angular 17, esbuild) did not require this flag.
11. **The finality-marking mechanism can produce zero MongoDB writes even when raw features are conceptually extracted.** AIRBUS extracted 4 raw features from the email input field, but the `FINALITY_SYSTEM_PROMPT` returned `[False, False, False, False]` — correctly determining that clicking a form field does not conclude any feature. Combined with the form submission failure, no action reached a finalized feature state, resulting in 0 MongoDB records. This reveals a general pattern: in applications where authentication blocks navigation and standard forms prevent successful submission, the pipeline may extract raw LLM features but store none of them — because no action can be identified as completing a feature end-to-end.

12. **The 30-action ceiling is the primary coverage bottleneck for applications with accessible, pre-seeded data.** For auth-blocked apps (EVERTRADUORA, AIRBUS), the authentication wall dominates before the ceiling can bind. HEROES had 53 total actions with complex per-record operations that were only partially explored within the ceiling.
13. **SOFTSCANNER_CQA run (2026-03-12):** 9 states, 9 transitions, 452 total candidate actions (30 executed), 42 raw LLM-inferred features, 6 features stored in MongoDB Atlas (finality=True), 9 screenshots. All 9 states share a single URL (`http://localhost:4200/`) — the pipeline distinguishes them by DOM hash. Angular Material tree toggles (ISO 25010 quality characteristics) generate distinct DOM states. No form submission occurred (Angular Material reactive forms lack `data-testid` attributes). The "Start Assessment" button was not reached within the 30-action ceiling. Binding constraint: 30-action ceiling.

1. Introduction

What AutoE2E presents itself as

The repository title is "AutoE2E: Feature-Driven End-To-End Test Generation." The README states:

> Source code and benchmark subjects for "AutoE2E: Feature-Driven End-To-End Test Generation."

The README's usage section instructs setting `APP_NAME` to "the name of the application you want to generate E2E test cases for," directly implying that running `python main.py` produces E2E test cases.

Why this evaluation was performed

This evaluation was conducted to:

- Determine whether the system produces executable E2E tests or only intermediate artifacts
- Document concrete outputs from six benchmark executions across varied application types and data states
- Compare pipeline behavior across authenticated vs. unauthenticated, seeded vs. empty, in-repository vs. external, and standard-form vs. custom-attribute applications
- Identify setup/execution gaps and pipeline assumptions violated by each benchmark
- Establish a factual multi-benchmark baseline covering pipeline generalizability across six application types

Questions this report answers

1. Does AutoE2E generate runnable E2E test cases?
2. What does the pipeline actually produce?
3. What did each benchmark run concretely discover?
4. What issues are encountered when running the pipeline on each benchmark?
5. How does authentication, data seeding, and form implementation affect coverage?
6. Is the pipeline extensible to external applications and different Angular versions?
7. What is the effect of full JWT authentication on both navigation coverage and MongoDB artifact production?

2. Evaluation Methodology

This evaluation combines three evidence sources:

Static code inspection

Every Python module in the `autoe2e/` package was read, along with `main.py` , all configuration files, all LLM prompts (`autoe2e/prompts.py`), the README, and benchmark application source files for all six apps.

Live pipeline execution — PETCLINIC (2026-03-10)

- Docker container for PetClinic REST API (port 9966)
- Angular 18 dev server (port 4200, `--serve-path /petclinic/`)
- Command: `python main.py` (with `APP_NAME=PETCLINIC` in `.env`)

Live pipeline execution — EVERTRADUORA (2026-03-12)

- Docker Compose (`docker-compose.demo.yaml` , `everco/ever-traduora:latest`) for Traduora + MySQL (port 8080)
- Command: `APP_NAME=EVERTRADUORA python main.py`

Live pipeline execution — POSTUSERS (2026-03-12)

- Backend: Express.js/TypeScript in-memory REST API at `http://localhost:5000` (`npm run dev` , `C:\Users\basha\git\github\posts-users-backend`)
- Frontend: Angular 15 SPA at `http://localhost:4200` (`npm start` , `C:\Users\basha\git\github\posts-users-ui-ng`)
- Command: `APP_NAME=POSTUSERS python main.py`
- Config: `configs/POSTUSERS.json` (created for this evaluation)

Live pipeline execution — HEROES (2026-03-12)

- Backend: JSON Server (`json-server --watch db.json --routes routes.json --port 7627`) providing seeded heroes and villains at `http://localhost:7627`
- Frontend: Angular 11 SPA at `http://localhost:7626` (`NODE_OPTIONS=--openssl-legacy-provider ng serve --proxy-config proxy.conf.json --port 7626` , `C:\Users\basha\git\github\heroes-angular`)
- Command: `APP_NAME=HEROES python main.py`
- Config: `configs/HEROES.json` (created for this evaluation)
- **Note**: `NODE_OPTIONS=--openssl-legacy-provider` was required due to Angular 11 build tooling incompatibility with Node.js 22's OpenSSL 3.x

Live pipeline execution — AIRBUS (2026-03-12)

- Database: MySQL 5.7 Docker container (`mysql:5.7` image pre-cached), port 3306; schema and user seed data applied via `docker exec`
- Backend: Spring Boot 2.5.5 at `http://localhost:8080` (`mvn spring-boot:run` , Java 17 runtime / Java 1.8 target), `C:\Users\basha\git\github\Inventory-Management-System\airbus-management-spring`
- Frontend: Angular 12.2.3 SPA at `http://localhost:4200` (`NODE_OPTIONS=--openssl-legacy-provider npx ng serve --port 4200` , `C:\Users\basha\git\github\Inventory-Management-System\AirbusInventory`)
- Command: `APP_NAME=AIRBUS python main.py`
- Config: `configs/AIRBUS.json` (created for this evaluation)
- **Note**: `NODE_OPTIONS=--openssl-legacy-provider` required for same OpenSSL reason as Angular 11 (HEROES). Angular 12 with `@angular-devkit/build-angular ~12.2.3` uses the same affected webpack APIs.

Live pipeline execution — SOFTSCANNER_CQA (2026-03-12)

- Backend: Node.js/TypeScript/Express at `http://localhost:3000` (`node dist/api/server.js` , pre-built — no compilation step required), `C:\Users\basha\git\softscanner\softscanner-continuous-quality-assessment-backend`
- Frontend: Angular 17.3 SPA at `http://localhost:4200` (`npx ng serve --port 4200` , `C:\Users\basha\git\softscanner\softscanner-continuous-quality-assessment-frontend`)
- Command: `APP_NAME=SOFTSCANNER_CQA python main.py`
- Config: `configs/SOFTSCANNER_CQA.json` (created for this evaluation)
- **Note**: Angular 17 uses esbuild (`@angular-devkit/build-angular:application`), not webpack. `NODE_OPTIONS=--openssl-legacy-provider` was **not** required. No Docker, no external database (SoftScanner's MongoDB telemetry DB is separate from AutoE2E's Atlas connection and is not required for the frontend to load).

Environment resets between runs

- EVERTRADUORA → POSTUSERS: stopped `traduora` + `mysqldb` Docker containers; killed PID holding port 4200
- POSTUSERS → HEROES: POSTUSERS processes had stopped; ports 4200, 5000, 7626, 7627 all confirmed free before HEROES startup
- After HEROES run: all heroes-angular processes stopped (json-server PID 1988, ng serve PID 19012, npm wrappers PIDs 16596, 24308); ports 7626 and 7627 confirmed free
- After AIRBUS run: Angular PID 24368 killed, Spring Boot PID 24560 killed, MySQL container `airbus-mysql` stopped and removed; ports 4200, 8080, 3306 all confirmed free
- Before SOFTSCANNER_CQA run: all target ports (3000, 4200) confirmed free; all Docker containers confirmed exited; no residual processes from prior runs
- After SOFTSCANNER_CQA run: Node.js backend (PID 720) and Angular frontend (PID 738) killed; ports 3000 and 4200 confirmed free

Artifact inspection

Post-execution artifacts for all six runs:

- `report/PETCLINIC.json` (30,366 bytes, 9 states)
- `report/EVERTRADUORA.json` (6,601 bytes, 3 states)
- `report/POSTUSERS.json` (95,654 bytes, 7 states)
- `report/HEROES.json` (16,813 bytes, 5 states)
- `report/AIRBUS.json` (1,995 bytes, 1 state)
- `report/SOFTSCANNER_CQA.json` (382,021 bytes, 9 states)
- 9 + 3 + 7 + 5 + 1 + 9 = 34 total screenshots across all runs
- MongoDB Atlas collections for all six runs (AIRBUS: 0 records — finality False for all features; SOFTSCANNER_CQA: 42 raw / 6 final)
- Full stdout/stderr logs: `/tmp/evertraduora_run4.log` , `/tmp/postusers_run.log` , `/tmp/heroes_run.log` , `/tmp/airbus-autoe2e.log` , `/tmp/softscanner-autoe2e.log`

3. Claimed Scope of the Approach

Repository title

> "AutoE2E: Feature-Driven End-To-End Test Generation"

This directly claims test generation.

README claims

- `APP_NAME` : "The name of the application you want to **generate E2E test cases for**"

baseline-prompts.md

Contains three prompt templates that explicitly instruct an LLM to generate Selenium-compatible test cases. **None of these prompts are executed by any code path in the repository.** Verified by searching all Python source files for any reference to `baseline-prompts.md` .

Documentation summary

The README describes: installing requirements, setting environment variables, running `python main.py` , LLM prompts, benchmark subjects, and a log-server. It does **not** describe a test-generation step, output test files, or any test framework integration.

4. Actual Implemented Pipeline

Stage 1: Initialization (`main.py:16-34` , `init_utils.py`)

- Deletes previous app data from MongoDB; reads config; initializes Chrome WebDriver
- Navigates to base URL; waits for SPA bootstrap (15s: `<app-root>` no longer contains "Loading")
- Extracts candidate actions; creates and enqueues initial state

Stage 2: State context extraction (`infer_utils.py:46-70`)

- Full-page screenshot → Claude Sonnet with `CONTEXT_EXTRACTION_SYSTEM_PROMPT` → 1-2 sentence page description

Stage 3: Action iteration (`main.py:60-154` , max 30 total)

- **Critical action detection** → Claude Haiku → True/False (irreversible actions skipped)
- **Action execution** → Selenium click or form fill+submit
- **State deduplication** → hash-based; new states enqueued
- **Feature extraction** → Claude Sonnet → `{probability, feature}` JSON array
- **Feature deduplication** → OpenAI embedding + vector search + Claude Sonnet similarity

- **Scoring and finality marking** → MongoDB records updated

Stage 4: Report generation (`main.py:161-194`)

Serializes state graph to `report/{APP_NAME}.json` . Closes WebDriver.

Where the pipeline stops

The pipeline terminates after writing the JSON state graph. No stage composes test scenarios, generates assertions, synthesizes test code, or writes `.spec` / `.test` / `.py` / `.js` files.

5. Does AutoE2E Actually Generate Executable Tests?

No. Unambiguously, no. Confirmed by static code analysis and six independent pipeline runs producing no test output.

Code-level evidence

1. `main.py:188-194` : Only output file is `report/{APP_NAME}.json` — a state graph, not test code.
2. `prompts.py` : All 6 LLM prompts extract descriptions or boolean values, not code.
3. No test framework imports exist for code-generation purposes.
4. No file-writing code writes `.spec` , `.test` , `.py` (test), `.js` , or `.ts` files.

Artifacts produced vs. tests

Artifact	Type	Executable as test?
`report/{APP_NAME}.json`	State graph (JSON)	No
MongoDB `functionality`	Feature descriptions + embeddings	No
MongoDB `action-functionality`	Action-feature mappings	No
Screenshots (`tmp/*.png`)	Visual evidence	No
Activity log	Execution trace	No

6. PETCLINIC Execution Environment and Setup

Required services

Service	Technology	Port	Purpose
PetClinic REST API	Docker (`webappdockers/petclinic-rest:latest`)	9966	Backend with seed data
PetClinic Angular frontend	Angular 18 dev server	4200	Web UI under test
MongoDB Atlas	Cloud	N/A	Feature/action storage
Chrome	ChromeDriver	N/A	Browser automation

Environment variables

```
APP_NAME=PETCLINIC
OPENAI_API_KEY=<required>
ANTHROPIC_API_KEY=<required>
ATLAS_URI=<required MongoDB Atlas connection string>
```

7. PETCLINIC Execution Issues Encountered

7.1 Angular base-href misconfiguration

`<base href="/petclinic">` (no trailing slash) → 404 on all JS/CSS bundles.
Fix: Changed to `<base href="/petclinic/">`.

7.2 Angular serve-path requirement

`ng serve` serves at `/`; config expects `/petclinic/`.
Fix: `MSYS_NO_PATHCONV=1 npx ng serve --port 4200 --serve-path /petclinic/`

7.3 Webpack glyphicons asset conflict

Two identical SVGs caused `Conflict: Multiple assets emit different content to the same filename`.
Fix: Replaced `src/assets/fonts/glyphicons-halflings-regular.svg` with the node_modules version.

7.4 SPA bootstrap timing

No Selenium wait → captured blank "Loading..." page → empty state graph.
Fix: Added `WebDriverWait` (15s) in `init_utils.py` and `crawl_context.py`.

7.5 npm peer dependency conflicts

`npm install` failed on Angular 18 / TypeScript 5.4 / Node.js 22.
Fix: `npm install --legacy-peer-deps`.

Summary of PETCLINIC documentation gaps

Issue	Documented?	Required?
Starting Docker Desktop	No	Yes
Docker container command	No	Yes
Angular npm install	No	Yes
`--serve-path` flag	No	Yes
base-href trailing slash	No	Yes
SPA wait timing	No	Yes
Git Bash path workaround	No	Yes (Windows)

8. Concrete PETCLINIC Outputs Obtained

File: `report/PETCLINIC.json` (30,366 bytes)

Metric	Value
Total states discovered	9
Total transitions	9
Total actions across all states	72
Unique URLs covered	7
30-action ceiling reached?	Yes

Metric	Value
FormAction paths traversed	4 (mechanically executable via PetClinic-specific `data-formid` / `data-testid` / `data-submitid` attrs; business-level submission outcomes not verified by pipeline)

Screenshots (9)

State	URL	File size
Homepage	`/petclinic/welcome`	121,991 bytes
Owners dropdown	`/petclinic/welcome`	115,906 bytes
Vets dropdown	`/petclinic/welcome`	119,693 bytes
Pet Types	`/petclinic/pettypes`	57,268 bytes
Specialties	`/petclinic/specialties`	53,706 bytes
Owners search	`/petclinic/owners`	92,896 bytes
Add Owner form	`/petclinic/owners/add`	54,863 bytes
Veterinarians list	`/petclinic/vets`	77,049 bytes
Add Veterinarian form	`/petclinic/vets/add`	39,681 bytes

MongoDB records

Collection	PETCLINIC docs
`functionality`	46
`action-functionality`	46

9. PETCLINIC Workflows / Features / Actions Discovered

State-by-state summary

State	URL	Actions	LLM Context
Homepage	`/petclinic/welcome`	6	"homepage of a veterinary clinic website"
Owners dropdown	`/petclinic/welcome`	8	"dropdown menu for pet owners"
Vets dropdown	`/petclinic/welcome`	8	"navigation dropdown for veterinarian sections"
Pet Types	`/petclinic/pettypes`	6	"pet types management page...edit or delete"
Specialties	`/petclinic/specialties`	6	"specialties management page...radiology, surgery, dentistry"
Owner search	`/petclinic/owners`	8	"search results page for owner database"
Add Owner	`/petclinic/owners/add`	13	"form for registering a new pet owner"
Vet list	`/petclinic/vets`	6	"veterinary clinic directory page"
Add Vet	`/petclinic/vets/add`	11	"form for adding a new veterinarian profile"

Observed workflow coverage

2-hop workflows reached: Owner management, Vet management, Owner search, Owner registration, Vet listing, Vet registration. Not reached within the 30-action ceiling: individual owner/pet detail pages, edit/delete flows for existing records, post-form-submission confirmation states.

10. EVERTRADUORA Execution Environment and Setup

Application description

Ever Traduora: Angular 12 + NestJS translation management platform. Served on port 8080 via Docker. All functionality requires authentication. Config: `configs/EVERTRADUORA.json` (`base_url: "http://localhost:8080/"`).

Required services

Service	Technology	Port
Traduora app	Docker (`everco/ever-traduora:latest`)	8080
MySQL	Docker (`mysql:5.7`)	3306

Startup command

```
cd benchmark/ever-traduora
docker-compose -f docker-compose.demo.yaml pull
docker-compose -f docker-compose.demo.yaml up -d
```

11. EVERTRADUORA Execution Issues Encountered

E1: `APP_NAME` environment override failure

`load_dotenv(override=False)` does not override shell-set env var. Pipeline loaded PETCLINIC config.
Fix: `APP_NAME=EVERTRADUORA python main.py` (explicit CLI override).

E2: Anthropic API 529 overload — no retry logic

Pipeline crashed on transient HTTP 529. `invoke_model_chain` had no retry logic.
Fix: 5-attempt exponential-backoff retry in `autoe2e/llm_api_call.py:create_model_chain` .

E3: `FORM_VALUE_SYSTEM_PROMPT` assumes `data-testid` attributes

LLM refused JSON for forms without `data-testid` → `json.loads()` crashed.
Fix: `try/except` in `create_form_filling_values` , returning `{}` on parse failure.

E4: `form_action.execute()` hangs on `input()` for non-PetClinic forms

Missing `data-formid` / `data-submitid` → exception → blocking `input()` call.
Fix: Removed `input()` call; replaced with log + silent return.

E5: Authentication wall

All functionality requires login; pipeline has no credential management.
Not fixed: Fundamental architectural constraint.

12. Concrete EVERTRADUORA Outputs Obtained

File: `report/EVERTRADUORA.json` (6,601 bytes)

Metric	Value
States discovered	3

Metric	Value
Transitions	3
Total actions	13
30-action ceiling?	No (13/30 used)
Forms submitted	0

Screenshots (3)

State	URL	File size
Login	`/login`	21,341 bytes
Sign up	`/signup`	23,697 bytes
Forgot password	`/forgot-password`	24,852 bytes

MongoDB records: 5 `functionality` docs (all from login page email input)

13. EVERTRADUORA: Navigation Surfaces, Forms, and LLM-Inferred Features

Observed navigation surfaces

`/login` (email + password inputs, sign up + forgot password links), `/signup` (name + email + password inputs), `/forgot-password` (email input + back link). No authenticated routes reachable.

LLM-inferred features (login page email input only)

Feature	Probability	Finality
"enter email address for authentication"	0.95	False
"validate email format input"	0.90	False
"display email validation error messages"	0.85	False
"interact with email input field"	0.75	**True**
"use email auto-complete suggestions"	0.40	False

All 5 features are LLM inferences from screenshot + HTML; no authentication workflow was executed.

14. POSTUSERS Execution Environment and Setup

Application description

"Posts & Users": Angular 15 + Express.js social platform with in-memory backend (no database). Starts empty — no posts or users pre-seeded. All routes public. Located at `C:\Users\basha\git\github\posts-users-ui-ng` (frontend) and `C:\Users\basha\git\github\posts-users-backend` (backend).

Config created: `configs/POSTUSERS.json` (`base_url: "http://localhost:4200/"`)

Startup commands

```
# Backend (port 5000)
cd C:\Users\basha\git\github\posts-users-backend && npm run dev

# Frontend (port 4200)
cd C:\Users\basha\git\github\posts-users-ui-ng && npm start
```

15. POSTUSERS Execution Issues Encountered

P1: Port 4200 occupied from prior session

Killed PID 35312 holding port 4200. **Classification**: Environment residue.

P2: Form submission incompatibility (E3/E4 already fixed)

Forms use `id=` attributes, no `data-testid`. Fix already applied — graceful skip with log.

P3: No seed data — dynamic routes unreachable

In-memory backend starts empty. `/users/:id` unreachable with no users.
Classification: Benchmark-specific architecture.

P4: 30-action ceiling as coverage bottleneck

`/new-user` alone has 38 actions — exceeds the entire budget.

16. Concrete POSTUSERS Outputs Obtained

File: `report/POSTUSERS.json` (95,654 bytes)

Metric	Value
States discovered	7
Transitions	7
Total actions across all states	95
Actions executed (ceiling)	30
30-action ceiling?	Yes (binding)
Forms submitted	0

Screenshots (7): `/posts` (24 KB), `/new-post` (33 KB), `/users` (24 KB), `/new-user` (48 KB), `/user-search` (31 KB), `/user-availability-schedule` (36 KB), `/user-search` variant (31 KB)

MongoDB records: ~37 raw features across 7 states

17. POSTUSERS: Navigation Surfaces, Forms, and LLM-Inferred Features

Observed navigation surfaces

URL	Content state	Actions
`/posts`	Empty list	6
`/new-post`	Post creation form	10
`/users`	Empty list	6
`/new-user`	38-field registration form	38
`/user-search`	Email search form	9
`/user-availability-schedule`	Scheduling form	17

Observed forms (none submitted)

New Post (`id="title"` , `id="content"`), New User (12 fields including toggles), User Search (email), Availability Schedule (date/time fields). All use standard HTML `id` ; no `data-testid` .

Notable features (LLM-inferred)

"create new post", "create new user account", "search for users", "schedule content publishing availability", "set user online availability schedule". Several inferences ("send user invitation", "view user statistics") are hallucinations — not implemented in this app.

18. HEROES Execution Environment and Setup

Application description

Heroes Angular is an Angular 11 Tour of Heroes implementation by John Papa (`C:\Users\basha\git\github\heroes-angular`). It is a full-stack demo application featuring:

- **Frontend**: Angular 11 SPA with NgRx/Data state management, Bulma CSS, Font Awesome; heroes and villains CRUD management
- **Backend**: JSON Server (`json-server`) with a `db.json` file providing **6 pre-seeded heroes and 4 pre-seeded villains** — a real REST API mock, not an in-memory Angular service
- **Routing**: `/heroes` (default), `/villains` , `/about` — lazy-loaded modules. Inline add/edit forms appear at the same `/heroes` URL with different DOM state
- **No authentication**: all routes fully public
- **State management**: NgRx/Data with HTTP entity services (`DefaultDataServiceConfig` targeting `environment.API = 'api'` , proxied to `http://localhost:7627`)

This application is **outside the `benchmark/` directory**. It was integrated for this evaluation via a single new config file.

Config created: `configs/HEROES.json` :

```
{
  "temp_dir": "./tmp",
  "driver": { "base_url": "http://localhost:7626/" },
  "lifecycle": {
    "on_visit": [],
    "on_state_discovery": [["autoe2e", "StateContextExtraction"]]
  }
}
```

Required services

Service	Technology	Port	Notes
JSON Server backend	`json-server` (`db.json`)	7627	6 heroes + 4 villains pre-seeded; auto-reset via `json-server-reset`
Angular frontend	Angular 11 dev server	7626	`ng serve --proxy-config proxy.conf.json --port 7626`
MongoDB Atlas	Cloud	N/A	Feature/action storage
Chrome	ChromeDriver	N/A	Browser automation

Startup commands

```
# Step 1: Install dependencies (first time only)
cd C:\Users\basha\git\github\heroes-angular
npm install --legacy-peer-deps

# Step 2: Start JSON Server backend (port 7627)
npm run backend
# = json-server --watch db.json --routes routes.json --port 7627 --middlewares ./node_modules/json-server-reset

# Step 3: Start Angular frontend (port 7626) — requires OpenSSL legacy flag
NODE_OPTIONS=--openssl-legacy-provider npm run start-ng
# = ng serve --proxy-config proxy.conf.json --port 7626
```

Proxy configuration

`proxy.conf.json` maps `/api/*` → `http://localhost:7627` . All Angular HTTP calls go via this proxy. NgRx/Data's `DefaultDataServiceConfig` uses `environment.API = 'api'` (relative path), so requests become `GET /api/heroes` → proxied → `GET http://localhost:7627/heroes` .

Angular 11 + Node.js 22 compatibility

Angular 11's webpack build uses OpenSSL APIs removed in Node.js 17+ (OpenSSL 3.x). This causes `ERR_OSSL_EVP_UNSUPPORTED` at build time. The `NODE_OPTIONS=--openssl-legacy-provider` flag restores the legacy OpenSSL provider for the build process only — it is a standard workaround for Angular versions ≤12 on Node.js ≥17.

JSON Server vs. in-memory Angular service

This application uses **json-server** — a real standalone process with HTTP endpoints, not Angular's `HttpClientInMemoryWebApiModule` . The distinction matters: json-server intercepts requests at the proxy level, while in-memory-web-api intercepts them inside the Angular `HttpClient` . Both provide similar end-user behavior but json-server means the Angular app is structurally identical whether testing locally or in production — the difference is only the backend implementation at the proxy target.

19. HEROES Execution Issues Encountered

Issue H1: Node.js 22 + Angular 11 OpenSSL incompatibility

Classification: Target-app environment (known compatibility issue)

Problem: `npm run start-ng` (`ng serve`) failed immediately with:

```
Error: error:0308010C:digital envelope routines::unsupported
code: 'ERR_OSSL_EVP_UNSUPPORTED'
Node.js v22.1.0
```

Angular 11 uses webpack 4/5 which calls OpenSSL APIs that were removed from Node.js 17+.

Fix: Restarted with `NODE_OPTIONS=--openssl-legacy-provider npm run start-ng` . No code changes required — environment variable only.

Impact on pipeline: The fix has zero effect on the running Angular application or the AutoE2E pipeline. Once the Angular dev server compiled successfully, the pipeline ran identically to any other Angular SPA.

Issue H2: `node_modules` not installed

Classification: Setup prerequisite (not documented)

Problem: The `heroes-angular` repository did not have `node_modules/` installed. `ng` binary was absent.

Fix: `npm install --legacy-peer-deps` (needed for Angular 11 + newer npm peer resolution).

Issue H3: No new AutoE2E pipeline issues

The HEROES run encountered **zero pipeline defects**. All three patches from EVERTRADUORA (retry logic, JSON error handling, form `input()` removal) were in place. Furthermore, HEROES' Angular reactive forms do not use `<form>` HTML elements in a way that the candidate action extractor detects as FormActions — all form interactions are click-typed actions — so the `data-testid` form issue was entirely avoided.

Summary of HEROES documentation gaps

Issue	Documented?	Impact
`NODE_OPTIONS=--openssl-legacy-provider` required	No	Blocking (build fails without it)
`npm install` needed first	No	Blocking
Both `npm run backend` and `npm run start-ng` must be started separately	Partially (README `quick` script does both)	Pipeline: must start both before running
`db.json` mutates during run (delete/edit actions)	No	JSON Server state drifts; run not deterministic without reset

20. Concrete HEROES Outputs Obtained

State graph report

File: `report/HEROES.json` (16,813 bytes)

Metric	Value
Total states discovered	5
Total transitions	5
Total actions across all states	53
Actions actually executed (ceiling)	30
Unique URLs covered	2 (`/heroes` × 3 states, `/villains` , `/about`)
30-action ceiling reached?	**Yes** (binding constraint)
Forms submitted	0 (no FormAction type detected in NgRx reactive forms)
API retries	0

Screenshots (5)

State	URL	File size	Content
Heroes list	`/heroes`	49,378 bytes	6 seeded hero cards with edit/delete buttons
Villains list	`/villains`	33,385 bytes	4 seeded villain cards

State	URL	File size	Content
About page	`/about`	85,252 bytes	Tutorial comparison (Angular/React/Vue)
Hero add form	`/heroes`	27,576 bytes	Inline form: name + description fields
Hero edit form	`/heroes`	30,686 bytes	Inline form: ID (readonly) + name + description

The heroes list screenshot (49 KB) shows 6 populated records with per-record action buttons — the richest list-page state across all six runs. The about page screenshot (85 KB, largest of the non-SPA screenshots) reflects the detailed comparison content.

MongoDB records

Collection	HEROES docs	Notes
`functionality`	~25 (after vector deduplication)	58 raw before dedup
`action-functionality`	~25	Action-to-feature mappings

The 13 feature-extraction calls (one per action that led to a new state) produced 58 raw feature strings, many of which are near-duplicate variants of "delete character from collection" extracted from the 6 hero delete buttons and 4 villain action buttons.

Execution summary

- Pipeline exit code: 0
- Approximate runtime: ~10 minutes
- No API overload retries
- No form submission attempts or failures

21. HEROES: Navigation Surfaces, Forms, and LLM-Inferred Features

Observed navigation surfaces (directly visited states)

State	URL	DOM variant	Actions	LLM-inferred context
Heroes list	`/heroes`	Default list	19	*"displays a list of heroes in a character management application, showing individual character profiles with their names, descriptions, and options to edit or delete each entry"*
Villains list	`/villains`	Default list	7	*"character management interface for villains...allowing users to view, edit, and delete villain character entries"*
About page	`/about`	Static content	4	*"information about a tutorial project...Angular, React, and Vue"*
Hero add form	`/heroes`	Add panel open	11	*"character creation form for adding new heroes...input fields for character name and description along with save and cancel options"*
Hero edit form	`/heroes`	Edit panel open	12	*"character editing interface...fields for ID, name, and description, with options to save or cancel changes"*

Key observation: Three distinct states share the URL `/heroes`. The pipeline correctly differentiated them by DOM state hash. This demonstrates that the state deduplication mechanism works correctly for single-page applications that show/hide UI panels without URL changes (a pattern common in NgRx-based Angular apps).

Not reached (within 30-action ceiling):

- Individual villain edit/add forms (villains state actions were partially consumed before revisiting)
- `**` (NotFoundComponent) — 404 route
- Any form submission → post-submission confirmation states
- Villain inline edit form (no edit button click within ceiling)

Observed interaction surface per state

Heroes list (19 actions):

- 4 navigation links (header brand + 3 nav items: heroes, villains, about)
- 1 "Add Hero" button opening the inline add form
- Per-hero buttons for 6 heroes: Edit (opens inline edit form) + Delete each hero
- Total: 4 nav + 1 add button + 6 × (edit + delete) = 19

Villains list (7 actions):

- 4 navigation links
- 3 villain action buttons (pipeline hit ceiling before processing per-villain edit/delete)

About page (4 actions):

- 4 navigation links only (static content page, no interactive elements)

Hero add form (11 actions):

- 4 nav + 1 add button + 3 list buttons (partial hero list still visible) + name input + description input + Save/Cancel buttons

Hero edit form (12 actions):

- 4 nav + 3 list buttons + ID input (readonly) + name input + description input + Save/Cancel + 1 additional list context button

Forms discovered (none submitted)

The hero add and edit forms are Angular reactive forms rendered inline within the heroes list component. They do **not** use standard HTML `<form>` element structures that the candidate action extractor (`CandidateActionExtractor`) registers as `FormAction` entries. All form inputs are registered as individual click-type actions. As a result:

- No `FORM_VALUE_SYSTEM_PROMPT` calls were made
- No `create_form_filling_values` failures
- No `form_action.execute()` issues
- Forms were interacted with only as click targets (individual field clicks), not as submitted forms

This is distinct from PETCLINIC (which has `<form data-formid=...>` wrapping inputs), EVERTRADUORA, and POSTUSERS (which have standard `<form>` elements). HEROES' inline reactive form rendering avoided the form-submission compatibility issue entirely.

LLM-inferred features (58 raw, ~25 unique after deduplication)

Features are grouped by the action context from which they were extracted. All are LLM inferences from screenshots and action HTML; none were verified by form submission or CRUD operation execution.

From navigation actions:

- "navigate to home page"
- "access application tour or tutorial"
- "reset to default character list view"
- "refresh character collection display"

From "Add Hero" button:

- "add new character to collection"
- "open character creation form"
- "initialize character creation workflow"
- "access character builder interface"
- "open character template selection"

From Edit button (one hero):

- "edit character details"
- "open character edit form"

- "navigate to character edit page"
- "enable character modification mode"

From Delete buttons (6 heroes × 3–5 features each, many near-duplicates):

- "delete character from collection"
- "trigger delete confirmation dialog"
- "remove character entry from display list"
- "update character collection count"
- "clear character selection state"
- "delete hero from character list"
- "show deletion confirmation dialog"
- "remove character from UI display"
- "update character count after deletion"
- "remove character data from data store"

Caveat on delete features: The 30 raw delete-related features are near-duplicate semantic variants extracted from 6 different hero delete buttons. Vector deduplication via OpenAI embeddings and Claude Sonnet similarity matching would collapse these into 4–6 unique features in MongoDB. The raw count (58) is misleadingly high relative to the semantic uniqueness (~25) of the feature set.

Critical action detection: Run behavior was consistent with critical-action filtering — delete buttons appear to have been flagged by the CRITICAL_ACTION_SYSTEM_PROMPT and skipped during execution (no hero records were deleted from `db.json` during the run). This behavior is appropriate: deleting heroes is irreversible in json-server without a reset. Per `main.py:70`, feature extraction is still attempted for critical actions (`should_extract_func = True`) even when execution is skipped, which accounts for the delete-related raw features in the feature set.

22. AIRBUS Execution Environment and Setup

Application description

Airbus Inventory Management System: Angular 12.2.3 + Spring Boot 2.5.5 (Java 1.8 bytecode target, Java 17 runtime) + MySQL 5.7 inventory CRUD platform. Located at `C:\Users\basha\git\github\Inventory-Management-System` (parent repo), with `AirbusInventory/` (frontend) and `airbus-management-spring/` (backend) subdirectories. All application features are protected by JWT authentication via `CanActivateRouteGuard`. Only the login page (`/login`) is accessible without credentials.

Config created: `configs/AIRBUS.json`:

```
{
  "temp_dir": "./tmp",
  "driver": { "base_url": "http://localhost:4200/" },
  "lifecycle": {
    "on_visit": [],
    "on_state_discovery": [["autoe2e", "StateContextExtraction"]]
  }
}
```

Required services

Service	Technology	Port	Notes
MySQL	Docker (`mysql:5.7`)	3306	External DB; requires manual schema creation + user seed
Spring Boot backend	Maven + Java 17	8080	Spring Boot 2.5.5, Spring JDBC, JWT auth
Angular frontend	Angular 12.2.3 dev server	4200	All CRUD routes auth-guarded
MongoDB Atlas	Cloud	N/A	Feature/action storage
Chrome	ChromeDriver	N/A	Browser automation

Startup commands

```
# Step 1: Start MySQL 5.7 Docker container (mysql:5.7 image pre-cached at 501MB)
docker run -d --name airbus-mysql \
  -e MYSQL_ROOT_PASSWORD=password \
  -e MYSQL_DATABASE=Product \
  -p 3306:3306 mysql:5.7

# Step 2: Wait for MySQL init (~5-10s), then create schema and seed users
docker exec airbus-mysql mysql -u root -ppassword Product -e "
  CREATE TABLE IF NOT EXISTS Product(productId varchar(256) UNIQUE NOT NULL,
    productName varchar(256), productDescription varchar(3500),
    productCategory varchar(256), units int);
  CREATE TABLE IF NOT EXISTS User(username varchar(256), password varchar(256));
  INSERT INTO User VALUES('airbus01','<bcrypt_hash>');
  INSERT INTO User VALUES('airbus02','<bcrypt_hash>');"

# Step 3: Start Spring Boot backend (port 8080)
cd C:\Users\basha\git\github\Inventory-Management-System\airbus-management-spring
mvn spring-boot:run   # Java 17 runtime; target bytecode 1.8

# Step 4: Install Angular dependencies (node_modules not in repo)
cd C:\Users\basha\git\github\Inventory-Management-System\AirbusInventory
npm install

# Step 5: Start Angular frontend (port 4200)
NODE_OPTIONS=--openssl-legacy-provider npx ng serve --port 4200
```

Application routing analysis

Route	Component	Auth-guarded?
`/`	Redirect → `/login`	No (redirect only)
`/login`	`LoginPageComponent`	No (public)
`/dashboard`	`GetAllProductsComponent`	**Yes** (`CanActivateRouteGuard`)
`/getAllProducts`	`GetAllProductsComponent`	**Yes**
`/productByCategory`	`GetProductByCategoryComponent`	**Yes**
`/add`	`AddProductComponent`	**Yes**
`/update`	`UpdateProductComponent`	**Yes**

`CanActivateRouteGuard` checks `localStorage.getItem('token')` . If absent (as it always is for the pipeline, which never authenticates), all protected routes redirect to `/login` . The pipeline can never reach the product dashboard or any CRUD view without an authenticated session.

Angular 12 + Node.js 22 compatibility

Same OpenSSL incompatibility as Angular 11 (HEROES benchmark). Angular 12's `@angular-devkit/build-angular ~12.2.3` uses webpack with OpenSSL APIs deprecated in Node.js 17+. `NODE_OPTIONS=--openssl-legacy-provider` was required. This confirms that the issue applies to Angular ≤12 on Node.js ≥17, regardless of the specific Angular major version.

CORS and Spring Security note

`WebSecurityConfig.configure(HttpSecurity)` calls `httpSecurity.cors()` without a `CorsConfigurationSource` bean. Spring Security's default CORS handling in this configuration would deny cross-origin requests from Angular at `localhost:4200` to the API at `localhost:8080` . This was noted during inspection but is irrelevant to the pipeline: AutoE2E crawls the Angular SPA and never makes direct HTTP calls to the backend API.

23. AIRBUS Execution Issues Encountered

Issue A1: JWT authentication wall — all routes except `/login` blocked

Classification: Target-app architecture (fundamental constraint)

Problem: ALL Angular routes except `/login` are protected by `CanActivateRouteGuard`, which checks `localStorage.getItem('token')`. The pipeline never authenticates; the token is always absent. All navigation attempts to `/dashboard`, `/getAllProducts`, `/productByCategory`, `/add`, or `/update` are silently redirected to `/login` by the guard.

Severity compared to EVERTRADUORA: More restrictive. EVERTRADUORA had 3 public pages (login, signup, forgot-password). AIRBUS has only 1 public page. This is the most auth-restricted benchmark in the set.

Not fixed: Fundamental architectural constraint — identical to EVERTRADUORA E5.

Issue A2: External MySQL database dependency

Classification: Target-app setup requirement

Problem: Unlike all prior benchmarks, this application requires a running MySQL server at `localhost:3306`. No Docker Compose file is provided for the full stack. The README provides SQL DDL statements but no automated startup. MySQL was not running on the evaluation machine; no MySQL CLI binary was present.

Workaround: Started MySQL 5.7 via Docker (`mysql:5.7` image was pre-cached at 501MB). Database schema (`Product` and `User` tables) and two user records with BCrypt-hashed passwords were inserted via `docker exec`. Spring Boot started successfully (HikariPool connected in < 1s). MySQL container stopped and removed after the run.

Impact on AutoE2E run: The backend starting correctly did not affect the pipeline outcome — the authentication wall remained binding regardless of whether the backend was operational.

Issue A3: Standard Angular Reactive Forms — FormAction detected but not submittable

Classification: AutoE2E pipeline limitation (previously documented as E3/E4)

Problem: The login form uses `[formGroup]="loginForm"` with `formControlName="emailid"` and `formControlName="password"` — standard Angular Reactive Forms bindings with no `data-testid`, `data-formid`, or `data-submitid` attributes. The `CandidateActionExtractor` correctly detected the `<form>` element as a FormAction. However, `create_form_filling_values` returned `{}` (E3 fix: LLM explicitly noted no `data-testid` attributes), and `form_action.execute()` failed silently when `@data-submitid='None'` produced `NoSuchElementException` (E4 fix: logged and returned without blocking).

Impact: Login could not be submitted. Authentication never achieved.

Issue A4: Finality marking prevents any MongoDB writes

Classification: Pipeline behavior observation (not a defect — correct behavior)

Observation: Four features were LLM-inferred from the email input field. All four received `finality = False` from `FINALITY_SYSTEM_PROMPT`. The LLM correctly determined that clicking/focusing an email input field does not itself conclude any feature (all four require typing, blur, or form submission to be "complete"). Because finality was `False` for all features, **no records were written to MongoDB** — the first run in this evaluation with zero MongoDB artifacts.

This reveals an important interaction: for auth-blocked applications with standard forms, not only is navigation blocked, but even the visible pre-auth form surface produces no concluded features. The pipeline's feature extraction runs, the LLM identifies conceptual features, but no feature can be "concluded" from a mere field click.

24. Concrete AIRBUS Outputs Obtained

File: `report/AIRBUS.json` (1,995 bytes)

Metric	Value
States discovered	1
Transitions	1 (self-loop: email input click → same state)
Candidate actions in state	3 (email input, password input, login form)
Total action-processing iterations	6 (3 actions processed on each of 2 revisits to the single state)
30-action ceiling reached?	No (6 / 30 used)
Forms detected (FormAction)	1 (login form, standard `<form>` element)
Forms submitted	0
Raw LLM features extracted	4 (from email input only)
Features stored in MongoDB	**0** (all finality = False)
API retries	0
Pipeline exit code	0
Approximate runtime	~5 minutes

Screenshot (1)

State	URL	File size	Content
Login page	`/login`	585,297 bytes	Email + password fields, "Sign In" button, Bootstrap card layout

MongoDB records: 0 `functionality` docs, 0 `action-functionality` docs

The zero-MongoDB result distinguishes AIRBUS from all prior runs. EVERTRADUORA (next-most-restricted) produced 5 functionality records because one feature ("interact with email input field") was marked `finality = True`. AIRBUS produced none: the FINALITY_SYSTEM_PROMPT returned `[False, False, False, False]` for all four email-field features, and the form action never completed feature extraction before failing.

What AutoE2E was unable to discover (behind the authentication wall, from source inspection only):

- Product list dashboard: tabular view of all products with sort, filter, and search capabilities
- Product creation form (`/add`): 5 fields — `productId` , `productName` , `productDescription` , `productCategory` , `units`
- Product update form (`/update`): same 5 fields, pre-populated for editing
- Category filter view (`/productByCategory`): products filtered by category dropdown
- Product deletion: HTTP DELETE endpoint per product; irreversible action
- Estimated 5–8 testable CRUD features if authentication were bypassed

25. AIRBUS: Navigation Surfaces, Forms, and LLM-Inferred Features

Observed navigation surfaces

State	URL	Actions	LLM-inferred context
Login page	`/login`	3	*"a login page for an Airbus Inventory Management System where users can enter their credentials to access the system"*

No additional states were discovered. Navigation to all authenticated routes redirected to `/login` by `CanActivateRouteGuard` .

Observed forms

Form	Location	Angular type	AutoE2E detects?	Submittable?
Login form	`/login`	Reactive Form (`[formGroup]` , `(ngSubmit)`)	**Yes** (FormAction)	**No** (no `data-testid` / `data-formid` / `data-submitid`)

This contrasts with HEROES: HEROES' NgRx reactive forms produced no `<form>` element that the extractor registered, so FormAction was never attempted. AIRBUS uses a standard `<form>` element with `[formGroup]` binding — the extractor correctly identifies it as a FormAction, but the submission subsystem cannot locate the required custom attributes.

LLM-inferred features (4 raw, 0 stored in MongoDB)

All extracted from the email input field (`id="inputEmail"` , `type="email"` , `formControlName="emailid"`):

Feature	Probability	Finality	Why False
"validate email address format and accept valid input"	0.9	**False**	Validation fires on blur or submit, not field click
"display validation error for invalid email format"	0.8	**False**	Error display requires typing + trigger
"enforce required field validation for empty email input"	0.7	**False**	Required validation triggers on submit attempt
"persist email input data during session"	0.3	**False**	Persistence requires typing text into the field

No features were extracted from the password input field or the login form (FormAction failed before feature extraction could be attempted for those).

Behind the authentication wall (source inspection only, not reached by pipeline)

From reading `app.module.ts` , `product.service.ts` , `add-product/` , `update-product/` , and `get-all-products/` source files:

Feature area	Route	Form fields	Auth required
Product list	`/dashboard` or `/getAllProducts`	None (table display)	Yes
Add product	`/add`	productId, productName, productDescription, productCategory, units	Yes
Update product	`/update`	Same 5 fields, pre-populated	Yes
Filter by category	`/productByCategory`	Category dropdown	Yes
Delete product	(button in list)	None (DELETE request)	Yes

None of these areas were reachable by the pipeline. The feature inventory above is derived from static code inspection, not AutoE2E execution — it is included to characterize the application scope that remained blocked.

26. SOFTSCANNER_CQA Execution Environment and Setup

Application overview

SoftScanner is a continuous quality assessment (CQA) platform for web applications. It maps abstract stakeholder quality goals (ISO/IEC 25010 characteristics) to observable telemetry metrics using the SoftScanner Quality Mapping Model (SSQMM). The platform instruments target applications dynamically, collects runtime telemetry, computes quality metrics, and streams live assessments via Server-Sent Events (SSE).

The frontend is an Angular 17.3 single-page application with one primary route (`/`). Its primary UI consists of a metadata input form (application name, codebase path, URL, type, technology) and a quality goal tree showing ISO 25010 characteristics (Interaction Capability, Functional Suitability, Performance Efficiency, Compatibility, Reliability) with collapsible sub-goals. Users fill in metadata, select quality goals, and start an assessment.

Infrastructure setup

Component	Details
Frontend	Angular 17.3 SPA, `softscanner-continuous-quality-assessment-frontend` , port 4200

Component	Details
Backend	Node.js/TypeScript/Express, `softscanner-continuous-quality-assessment-backend` , port 3000
Frontend command	`npx ng serve --port 4200`
Backend command	`node dist/api/server.js` (pre-built — no compilation required)
Database	None required at startup (SoftScanner's own MongoDB telemetry DB is separate from AutoE2E's Atlas connection)
Auth	None
Config file	`configs/SOFTSCANNER_CQA.json` (created for this evaluation)
Angular build tooling	esbuild (`@angular-devkit/build-angular:application`) — no `NODE_OPTIONS=--openssl-legacy-provider` needed

Key structural characteristics

- **Single URL**: all UI states share `http://localhost:4200/` ; distinct DOM configurations are created by expanding ISO tree nodes, opening Angular Material `mat-select` dropdowns, and clicking the "View Details" info button
- **Angular Material components**: `mat-form-field` , `mat-select` , `mat-tree` , `mat-checkbox` , `mat-card` , `mat-icon-button` — rich component library creating numerous interactive elements
- **No authentication**: all parts of the UI are publicly accessible
- **No seed data**: the quality model is hardcoded in the backend service, not loaded from a database
- **Angular reactive forms**: `<form [formGroup]="...">` with `formControlName` bindings; standard `<input>` elements with `id` and `name` attributes but no `data-testid` , `data-formid` , or `data-submitid` attributes

Config created

```
{
  "temp_dir": "./tmp",
  "driver": {
    "base_url": "http://localhost:4200/"
  },
  "lifecycle": {
    "on_visit": [],
    "on_state_discovery": [["autoe2e", "StateContextExtraction"]]
  }
}
```

27. SOFTSCANNER_CQA Execution Issues Encountered

No blocking errors

The SOFTSCANNER_CQA run completed without pipeline crashes, LLM API errors, or blocking input prompts. This is the cleanest external-app run of the six benchmarks from an operational standpoint.

Angular 17 esbuild compatibility

Angular 17.3 uses `@angular-devkit/build-angular:application` with esbuild as the build backend. Unlike Angular 11/12 (webpack-based), Angular 17 does not trigger the OpenSSL 3.x incompatibility issue observed in HEROES and AIRBUS. No `NODE_OPTIONS` workaround was needed. The dev server started on first attempt.

Backend pre-built state

The SoftScanner backend's `dist/api/server.js` was already compiled. Running `node dist/api/server.js` directly avoided the TypeScript compilation step from `npm start` . The backend API responded correctly (`GET /api/quality-model` returned the ISO 25010 model JSON).

MongoDB port disambiguation

The SoftScanner backend's README mentions a MongoDB telemetry database at `localhost:27017` (database `continuous-quality-assessment-web-telemetry-mongodb`). This is distinct from AutoE2E's MongoDB Atlas connection (`ATLAS_URI` in `.env`). The AutoE2E pipeline reads and writes only to Atlas (`myDatabase`); the SoftScanner telemetry MongoDB was not started and is not required for the frontend to render.

No new pipeline defects discovered

All previously identified defects (E2–E4, SPA bootstrap fix) handled the run correctly. The form-submission defect (E3b) was present but the "Start Assessment" button was not reached within the 30-action ceiling, so no form submission was attempted.

28. Concrete SOFTSCANNER_CQA Outputs Obtained

State graph

`report/SOFTSCANNER_CQA.json` — 382,021 bytes

Metric	Value
States discovered	9
Transitions	9 (all from initial state)
Candidate actions (total across all states)	452
Actions executed	30 (30-action ceiling reached)
Unique URLs	1 (`http://localhost:4200/`)
Screenshots	9 (4 KB – 58 KB)

All 9 states are at the same URL. The pipeline distinguishes them by DOM content hash. The high candidate action count (452) reflects Angular Material's verbose component tree — each `mat-form-field` , `mat-select` , `mat-tree-node` , `mat-checkbox` , and `mat-icon-button` generates multiple clickable elements.

State descriptions (LLM-assigned contexts)

State hash (first 12)	LLM context
`233733720c0e`	"web application form for configuring continuous quality assessment, allowing users to input application metadata including name, codebase path, URL, type, and technology selections"
`f0c0cbd7fa6e`	"form for configuring continuous quality assessment, where users can select application types and set up quality mapping model parameters"
`33f5c8d0325c`	"form for configuring continuous quality assessment settings, where users can select technology options and define quality mapping model parameters"
`7cced15a79ae`	"form for configuring continuous quality web assessment, where users can input application details and toggle quality tree nodes"
`af9ccb44c329`	"detailed information or help page appearing after clicking a 'View Details' button"
`8203b80be480`	"form for setting up continuous quality web assessment" (Functional Suitability tree expanded)
`e9d69921d8e0`	"form for setting up continuous quality web assessment" (Performance Efficiency tree expanded)
`693712233783`	"form for configuring continuous quality web assessment" (Compatibility tree expanded)
`fd9a4e95ca52`	"form for configuring continuous quality web assessment" (Reliability tree expanded)

Features (MongoDB Atlas)

Metric	Value
Raw LLM-extracted features	42
Features stored (finality=True)	6
Features deduplicated (semantic)	Multiple duplicates: "navigate to next step in form wizard" extracted 5 times (one per ISO characteristic toggle button)

6 stored features:

1. `display available application type options` (Type mat-select click)
2–6. `navigate to next step in form wizard` × 5 (Interaction Capability, Functional Suitability, Performance Efficiency, Compatibility, and Reliability chevron toggle buttons)

Execution log

`/tmp/softscanner-autoe2e.log` — 1,080 lines. Pipeline ran cleanly; 30 actions executed; 9 new states discovered; 42 features inserted into Atlas; 6 marked as final. Exit code 0.

29. SOFTSCANNER_CQA: Navigation Surfaces, Forms, and LLM-Inferred Features

Navigation surface

The application has a single route (`/`) rendered as a full-page Angular Material card layout. The pipeline discovered 9 DOM states at this URL:

1. **Initial form state** — metadata form (Name, Codebase Path, URL, Type dropdown, Technology dropdown) + quality goal tree with ISO 25010 top-level nodes (Interaction Capability, Functional Suitability, Performance Efficiency, Compatibility, Reliability) collapsed
2. **Type dropdown open** — `mat-select` for "Type" field expanded (new DOM with overlay panel)
3. **Technology dropdown open** — `mat-select` for "Technology" field expanded
4. **Interaction Capability tree expanded** — chevron button clicked, sub-goals visible
5. **View Details info modal** — info icon button clicked, contextual help panel visible
6. **Functional Suitability tree expanded**
7. **Performance Efficiency tree expanded**
8. **Compatibility tree expanded**
9. **Reliability tree expanded**

All transitions originate from the initial state (state `233733720c0e`), reflecting that the pipeline always reloads the base URL between actions.

Form behavior

The main form uses Angular reactive forms (`[formGroup]="metadataForm"`) with Angular Material `mat-form-field` wrappers. The `<input>` elements have `id` and `name` attributes (e.g., `id="name"` , `id="path"` , `id="url"`) but no `data-testid` , `data-formid` , or `data-submitid` attributes.

The pipeline's `CandidateActionExtractor` did not detect the metadata form as a `FormAction` type (Angular Material's component tree does not produce the `<form>` element structure expected by the extractor). All form fields were treated as individual click/input actions. No form submission was attempted in this run.

The "Start Assessment" button (`mat-raised-button`) appears at the bottom of the quality goal panel. It was not reached within the 30-action ceiling, which was exhausted on form field interactions and tree node expansions.

Quality model tree

The ISO 25010 quality model tree is rendered using `mat-tree` with `mat-tree-node` leaf nodes. Each top-level node has:

- A `mat-icon-button` toggle (chevron_right icon, `aria-label="toggle <Characteristic>"`)
- A `mat-checkbox` for goal selection
- An `info` icon button (`matTooltip="View Details"`) opening a detail panel

The pipeline executed chevron toggles for all 5 top-level ISO characteristics (30 of the 452 candidate actions were spent on form field clicks and chevron toggles before the ceiling was reached). The `mat-checkbox` elements and "Start Assessment" button were not executed.

LLM feature extraction behavior

The pipeline extracted 5 features per action for each of the 9 actions that led to new states. Most features are functionally similar or identical across different tree toggle actions:

Extracted feature	Frequency	Final?
`navigate to next step in form wizard`	5×	**Yes** (all 5)
`display available application type options`	1×	**Yes**
`enter application name for quality assessment setup`	1×	No
`validate required application name field`	1×	No
`select application type from dropdown`	1×	No
`open dropdown menu for selection`	5×	No
`expand section to show additional form fields`	5×	No
`toggle panel visibility`	5×	No
`proceed with form submission`	5×	No
Other (help/info, tech/type validation)	~8	No

The heavy duplication (5× "navigate to next step in form wizard" — one per ISO characteristic toggle) is characteristic of the pipeline's per-action LLM feature extraction when the same action type appears multiple times in the UI with similar semantic context. Vector deduplication via Atlas `$vectorSearch` did not collapse these into one record because each was inserted independently before any deduplication threshold could be applied.

Form compatibility assessment

Attribute	Present in SoftScanner?
`data-testid` on inputs	No
`data-formid` on form	No
`data-submitid` on submit	No
Standard `<input>` elements	Yes
Angular Material `mat-select`	Yes
Standard `<form>` element	Yes (one metadata form)
Reactive form bindings	Yes (`formControlName`)
FormAction detected by pipeline	No (Angular Material structure not recognized)
Form submitted by pipeline	No

30. Cross-Benchmark Comparison

Benchmark comparison table

Property	PETCLINIC	EVERTRADUORA	POSTUSERS	HEROES	AIRBUS	SOFTSCANNER_C
Location	`benchmark/`	`benchmark/`	External (`posts-users-*`)	External (`heroes-angular`)	External (`Inventory-Management-System`)	External (`softscanner-cqa-*`

Property	PETCLINIC	EVERTRADUORA	POSTUSERS	HEROES	AIRBUS	SOFTSCANNER_C
Application type	Vet clinic (Angular 18)	Translation mgmt (Angular 12 + NestJS)	Social platform (Angular 15 + Express)	CRUD demo (Angular 11 + JSON Server)	Inventory CRUD (Angular 12 + Spring Boot 2.5 + MySQL)	Quality assessment platform (Angular 17 Node.js/Express)
Setup method	Docker + Angular dev server	Docker Compose (pre-built)	Direct `npm` (2 repos)	`npm` + JSON Server (1 repo)	Docker MySQL + `mvn` + `npm` (2 repos)	Direct `node` + `ng serve` (2 repos, pre-built)
Setup complexity	High (5 issues, 7 undoc. steps)	Medium (4 issues, 5 undoc. steps)	Low (1 port conflict, 3 undoc. steps)	Low (2 issues: OpenSSL + npm install)	Medium (MySQL container + Maven + Angular, 4 undoc. steps)	**Lowest (0 issues, clean start)**
AutoE2E config	Pre-existing	Pre-existing	`POSTUSERS.json` (created)	`HEROES.json` (created)	`AIRBUS.json` (created)	`SOFTSCANNER_CQA.js` (created)
Auth required	No	Yes (all features)	No	No	**Yes (all features, JWT)**	No
`data-testid` on forms	Yes (custom PetClinic)	No	No	N/A (no FormAction detected)	No (standard `formControlName`)	No (Angular Materia reactive form)
Pre-seeded / live data	Yes (Docker REST API)	No (auth-blocked)	No (in-memory, empty)	**Yes (db.json, 6+4 records)**	No (auth-blocked; MySQL empty Product table)	N/A (quality model is hardcoded in backe
Data layer	Docker container	MySQL + NestJS	In-memory arrays	JSON Server file	MySQL 5.7 (Docker) + Spring JDBC	No external data lay
States discovered	**9**	**3**	**7**	**5**	**1**	**9**
Transitions	**9**	**3**	**7**	**5**	**1 (self-loop)**	**9 (all from initial state)**
Total candidate actions	**72**	**13**	**95**	**53**	**3**	**452**
Actions executed	**30**	**13 (all)**	**30**	**30**	**6 (3 per revisit × 2)**	**30**
Raw LLM features extracted	**46**	**5**	**37**	**58 (~25 unique)**	**4**	**42**
Features stored in MongoDB	**46**	**5**	**~37**	**~25**	**0**	**6**
Screenshots	**9**	**3**	**7**	**5**	**1**	**9**
Report file size	**30,366 bytes**	**6,601 bytes**	**95,654 bytes**	**16,813 bytes**	**1,995 bytes**	**382,021 bytes**
Binding constraint	30-action ceiling	Auth wall	30-action ceiling	30-action ceiling	**Auth wall + finality=False**	30-action ceiling

Property	PETCLINIC	EVERTRADUORA	POSTUSERS	HEROES	AIRBUS	SOFTSCANNER_C
FormAction paths traversed	4 (PetClinic-specific attrs; outcomes not verified)	0	0	N/A	0	0
FormAction detected	Yes	Yes	Yes	No (`<form>` not detected)	**Yes** (login form, standard `<form>`)	No (Angular Materia structure not recognized)
Unique URL depth reached	7 URLs, 2-hop	3 URLs, 1-hop	6 URLs, 1-hop	2 URLs, 3 states/URL	**1 URL, 0-hop**	**1 URL, 0-hop (SPA, DOM states)**
Inline DOM state changes	No	No	No	**Yes (3 states at `/heroes`)**	No	**Yes (9 DOM states `/`)**
Notable areas reached	Owner/vet mgmt, add forms	Login/signup/forgot-password	All 6 routes; forms documented	Heroes/villains with seeded data, add/edit forms	Login page only	Metadata form, type/technology dropdowns, ISO 250 quality tree nodes, details panel
Notable unreached	Owner/pet detail, edit/delete	All authenticated features	User profiles, form submissions	Villain forms, form submissions	**All** (product dashboard, CRUD, category filter)	"Start Assessment" button, goal checkboxes, assessment results view
Node.js compat issue	No	No	No	Yes (Angular 11 + Node 22)	**Yes (Angular 12 + Node 22, same fix)**	**No (Angular 17 esbuild, no OpenS flag needed)**
External database dependency	Docker container	MySQL in Docker Compose	None	None	**MySQL 5.7 (separate Docker container)**	None
~Runtime	~14 min	~10 min	~10 min	~10 min	~5 min	~13 min

Benchmark-contained apps vs. external local apps

All six benchmarks are now categorized:

In-repository benchmarks (`benchmark/`): PETCLINIC and EVERTRADUORA ship with the AutoE2E repo, alongside Docker scripts and matching configs. They represent the "intended" use case. Setup friction for PETCLINIC was highest of all six runs (5 issues). EVERTRADUORA demonstrated that even intended benchmarks can be severely blocked if application architecture (authentication, standard HTML) doesn't match pipeline assumptions.

External apps: POSTUSERS, HEROES, AIRBUS, and SOFTSCANNER_CQA were not included in the repository. Integration required only creating a new config file in `configs/` . No other AutoE2E changes were needed for any of the four. This confirms that the config-file integration path is the intended and sufficient mechanism for new targets — extensibility is at the configuration level only, not at the workflow or authentication level. SOFTSCANNER_CQA was the easiest to start of all six (Angular 17, pre-built backend, no Docker, no seed data, no auth).

Benchmark categories and their effect on AutoE2E coverage

Category	Examples	Effect on coverage
Benchmark-contained, pre-seeded, no auth	PETCLINIC	Broadest coverage in this evaluation: 9 states, 7 URLs, 46 features stored — enabled by pre-seeded REST API and custom `data-testid` instrumentation (this app only)

Category	Examples	Effect on coverage
Benchmark-contained, auth-required	EVERTRADUORA	Severely limited: auth wall limits coverage to pre-auth surface; 3 public pages
External, empty backend, no auth	POSTUSERS	Moderate: all routes reachable but no record-level states; forms found but not submitted
External, seeded backend, no auth	HEROES	Good: list pages populated → per-record states; inline form states captured
External, JWT-protected, external DB	AIRBUS	**Worst**: single public page, FormAction detected but not submitted, 0 MongoDB records
External, no auth, SPA (single route, Angular Material)	SOFTSCANNER_CQA	Moderate: 9 DOM states at 1 URL (tree toggles + dropdowns), 42 raw features, only 6 stored; ceiling consumed by UI widget interactions before "Start Assessment" reached

Key finding: The most important predictor of coverage quality is not whether the app is in `benchmark/` but whether (a) the application is publicly accessible without authentication and (b) backend data exists at crawl time. HEROES (external, json-server seeded) achieved coverage comparable to PETCLINIC (in-repository, Docker-seeded). AIRBUS (external, full JWT authentication) produced the smallest artifact set of all six runs — smaller than EVERTRADUORA because AIRBUS has only one public page (no signup/forgot-password equivalents) and the finality mechanism correctly returned `False` for all conceptual features, resulting in zero MongoDB writes. SOFTSCANNER_CQA revealed a new pattern: a publicly accessible SPA with a highly interactive Angular Material UI can exhaust the 30-action ceiling on widget-level interactions (dropdown opens, tree node expansions) before reaching the primary application workflow entry point (the "Start Assessment" button).

Form compatibility patterns

App	Form detection	Submission	Notes
PETCLINIC	FormAction (via `<form data-formid=...>`)	Mechanically traversed (PetClinic-specific attrs; business outcomes not verified)	PetClinic-instrumented HTML
EVERTRADUORA	FormAction (via `<form>` HTML element)	No (missing `data-testid`)	Standard HTML form
POSTUSERS	FormAction (via `<form>` HTML element)	No (missing `data-testid`)	Standard HTML form
HEROES	No FormAction (NgRx reactive forms, no `<form>` wrapper detected)	N/A	Accidental bypass
AIRBUS	**FormAction** (via `<form>` with `[formGroup]`)	**No** (missing `data-testid`)	Angular Reactive Form, standard `<form>`
SOFTSCANNER_CQA	No FormAction (Angular Material `mat-form-field` + `mat-select` structure not recognized by `CandidateActionExtractor`)	No	Angular Material reactive form, `id` / `name` attrs present, no `data-testid`

HEROES uniquely avoided the form-submission issue entirely because its NgRx reactive form rendering does not produce the `<form>` element structure that `CandidateActionExtractor` recognizes as a FormAction. AIRBUS uses the same `[formGroup]` directive but with a standard `<form>` element — it was correctly detected as a FormAction and correctly failed to submit. The underlying pipeline limitation (requiring `data-testid`) remains the dominant form compatibility constraint across 4 of the 5 benchmarks.

Coverage progression across six runs

PETCLINIC → EVERTRADUORA → POSTUSERS → HEROES → AIRBUS → SOFTSCANNER_CQA represents variation across: (a) in-repository vs. external, (b) seeded vs. empty data, (c) authenticated vs. public, (d) form-instrumented vs. standard, (e) self-contained vs. external-database-dependent, and (f) single-URL SPA with high-density Angular Material UI. The six runs together demonstrate that pipeline coverage is primarily determined by application architecture — specifically authentication model, data availability, and action-density of the UI framework — not pipeline configuration or application location.

31. Value of the Produced Artifacts

PETCLINIC

Strongest artifact set: 9-state graph, 46 features with embeddings, 72 actions with `data-testid` locators, 4 form structures with field specifications. Directly usable as input to a downstream test-generation system.

EVERTRADUORA

Minimal artifact set: 3 pre-auth pages, 5 features from a single action. Limited practical value.

POSTUSERS

Moderate artifact set: 7 states, 37 raw features, rich form HTML (38-field new-user form documented completely). No verified interactions. The form specification is the most complete form artifact of the six runs.

HEROES

Good artifact set: 5 states capturing both list and inline form variants, ~25 deduplicated features covering hero/villain CRUD semantics, and run behavior consistent with critical-action filtering on delete buttons. The per-record action detection (6 heroes × edit+delete) demonstrates that seeded data enables meaningful action granularity.

AIRBUS

Empty MongoDB artifact set: 1-state graph (login page), 1 screenshot, 0 functionality records, 0 action-functionality records. The state graph and screenshot are the only outputs. Practical value as an AutoE2E artifact: minimal. Value as an empirical data point: significant — confirms that full JWT authentication completely prevents feature storage, not just navigation depth.

SOFTSCANNER_CQA

Partial artifact set: 9-state graph (all at one URL), 9 screenshots, 42 raw functionality records, 6 final functionality records. The 6 stored features are primarily "navigate to next step in form wizard" repeated for each ISO quality characteristic toggle — near-duplicate but technically distinct MongoDB documents. The quality model tree structure (ISO 25010 hierarchy) was documented as distinct DOM states. Practical value: moderate — the state graph captures distinct DOM configurations of a complex Angular Material SPA. The high candidate action count (452) correctly reflects the widget density of Angular Material components. The "Start Assessment" workflow was not reached, limiting usefulness for downstream test generation for the application's primary function.

Collective value boundary

All artifact sets are intermediate data — verified navigation maps, LLM-inferred feature inventories, action-element locators — useful to a downstream test-generation system that does not yet exist in this repository. AIRBUS demonstrates the minimum observed artifact set in this evaluation for a fully auth-blocked application with standard forms. SOFTSCANNER_CQA demonstrates that even a fully accessible SPA can have low practical feature coverage when the 30-action ceiling is exhausted on UI widget interactions before the primary workflow entry point is reached.

32. Gap Analysis

Claimed vs. implemented (across all six runs)

Stage	Claimed	Implemented	Evidence
Web application crawling	Yes	Yes	9/3/7/5/1/9 states across all runs
State discovery	Yes	Yes	All state graphs produced
Action extraction	Yes	Yes	72/13/95/53/3/452 actions
Page context summarization	Yes	Yes	LLM context per state, all six runs
Feature extraction (raw LLM output)	Yes	Yes	46/5/37/58/4/42 raw features across six runs

Stage	Claimed	Implemented	Evidence
Feature storage (MongoDB Atlas, after finality check)	Yes	Yes	46/5/$_{37/}$25/0/6 records stored (AIRBUS: 0 — all finality=False; SOFTSCANNER_CQA: 6 of 42)
Feature deduplication	Yes	Yes	Vector search + semantic matching
Action-feature mapping	Yes	Yes	MongoDB Atlas records
Test scenario composition	Implied	**Not implemented**	No code path in any run
Assertion generation	Implied	**Not implemented**	No code path in any run
Test code synthesis	Implied	**Not implemented**	No code path in any run
Test file emission	Implied	**Not implemented**	No file writing
Authentication handling	Not mentioned	**Not implemented**	EVERTRADUORA, AIRBUS: auth wall
Standard form compatibility	Not mentioned	**Not implemented**	Requires PetClinic `data-testid` ; Angular Material forms also incompatible (SOFTSCANNER_CQA)
Angular Material form detection	Not mentioned	**Not implemented**	`CandidateActionExtractor` does not recognize `mat-form-field` / `mat-select` as FormAction
API retry resilience	Not mentioned	Added via fix	Retry in `llm_api_call.py`
External app extensibility	Not documented	**Works**	POSTUSERS + HEROES + AIRBUS + SOFTSCANNER_CQA via config-only
Multi-Node.js-version compat	Not documented	**Partial**	Angular ≤12 needs OpenSSL env var; Angular 17 (esbuild) does not

33. Evidence Register

Source / Artifact	Type	What It Supports	Reliability / Limitations
`report/PETCLINIC.json` (30,366 bytes)	State graph	9 states, 72 actions	Direct; bounded by 30-action cap
`report/EVERTRADUORA.json` (6,601 bytes)	State graph	3 pre-auth states	Direct; bounded by auth wall
`report/POSTUSERS.json` (95,654 bytes)	State graph	7 states, 95 total actions	Direct; bounded by 30-action cap
`report/HEROES.json` (16,813 bytes)	State graph	5 states, 53 total actions	Direct; bounded by 30-action cap
`report/AIRBUS.json` (1,995 bytes)	State graph	1 state (login), self-loop edge	Direct; bounded by auth wall

Source / Artifact	Type	What It Supports	Reliability / Limitations
`report/SOFTSCANNER_CQA.json` (382,021 bytes)	State graph	9 states (all at `http://localhost:4200/`), 452 candidate actions	Direct; bounded by 30-action cap
9 PETCLINIC screenshots (39–122 KB)	Visual	Pages rendered correctly	Direct
3 EVERTRADUORA screenshots (21–25 KB)	Visual	Pre-auth pages only	Direct
7 POSTUSERS screenshots (24–48 KB)	Visual	All 7 routes rendered	Direct
5 HEROES screenshots (27–85 KB)	Visual	Heroes/villains with seeded data, inline forms	Direct
1 AIRBUS screenshot (585 KB)	Visual	Login page only	Direct
9 SOFTSCANNER_CQA screenshots (4–58 KB)	Visual	9 DOM states: metadata form, 2 dropdowns open, 5 quality tree nodes expanded, details panel	Direct
MongoDB Atlas `functionality` (46/5/$_{37/}$25/0/42 docs across 6 runs; 6 SOFTSCANNER_CQA final)	Feature records	LLM-inferred features with scores	LLM inference; not verified by execution
MongoDB Atlas `action-functionality` (46/5/$_{37/}$25/0/42 docs)	Action-feature maps	Action-to-feature linkage	LLM inference
`/tmp/evertraduora_run4.log`	Execution trace	Full EVERTRADUORA issues + LLM calls	Complete
`/tmp/postusers_run.log`	Execution trace	Full POSTUSERS run	Complete
`/tmp/heroes_run.log`	Execution trace	Full HEROES run; 13 feature-extraction calls	Complete
`/tmp/airbus-autoe2e.log`	Execution trace	Full AIRBUS run; finality=False for all features	Complete
`autoe2e/prompts.py`	Static code	Pipeline never generates test code	Definitive
`main.py` (195 lines)	Static code	Pipeline terminates at JSON; no test output	Definitive
`form_action.py`	Static code	Form submission requires PetClinic attributes	Definitive
`configs/POSTUSERS.json` + `configs/HEROES.json` + `configs/AIRBUS.json`	Configs (created)	External app config-file integration	Authoritative
`heroes-angular/db.json`	Seed data file	6 heroes + 4 villains seeded for run	Authoritative
`heroes-angular/proxy.conf.json`	Proxy config	`/api` → `http://localhost:7627`	Authoritative
`heroes-angular/package.json` `start-ng` script	Source	Port 7626, proxy config, `--openssl-legacy-provider` needed	Authoritative
`heroes-angular/src/app/store/config.ts`	Source	NgRx entity URLs via `environment.API = 'api'`	Authoritative
`AirbusInventory/src/app/app.module.ts`	Source	Angular routes, auth guard on all non-login routes	Authoritative
`AirbusInventory/src/app/can-activate-guard.ts`	Source	JWT auth guard: `localStorage.getItem('token')`	Authoritative

Source / Artifact	Type	What It Supports	Reliability / Limitations
		check	
`AirbusInventory/src/app/services/product.service.ts`	Source	All API calls hardcoded to `localhost:8080`, JWT-protected	Authoritative
`airbus-management-spring/src/main/resources/application.properties`	Source	MySQL at `localhost:3306/Product`, port 8080	Authoritative
`airbus-management-spring/pom.xml`	Source	Spring Boot 2.5.5, Java 1.8 target, MySQL + H2 runtime deps	Authoritative
Search across `AirbusInventory/src/` for `data-testid`	Source inspection	No `data-testid` in Angular frontend	Authoritative
`baseline-prompts.md`	Documentation	Test-generation prompts not executed by any code	Verified by codebase grep

34. Threats to Validity / Evaluation Limits

Exploration cap

`main.py:63`: `if LOOP_COUNTER > 30: break`. Binding for PETCLINIC, POSTUSERS, HEROES, and SOFTSCANNER_CQA. HEROES' heroes list alone has 19 actions (6 heroes × edit+delete + nav + add button); with 5 states total, many per-record interactions in the villains list were unreached within the ceiling. For AIRBUS, the cap was not binding (only 6 executions total) — the auth wall was the dominant constraint. For SOFTSCANNER_CQA, the 452 candidate actions (across 9 Angular Material-heavy states) make the ceiling particularly restrictive — the primary workflow entry point ("Start Assessment" button) was never reached.

Benchmark-specific attribute dependency

`form_action.py:43` requires `@data-testid` XPath for form fields and `@data-submitid` for submit buttons. This is non-standard. Only PetClinic's templates include these attributes. HEROES avoided this entirely because its NgRx reactive forms were not detected as FormAction type. AIRBUS confirmed the pattern: the login form was detected as FormAction but could not be submitted. SOFTSCANNER_CQA adds a new variant: Angular Material `mat-form-field` / `mat-select` components are not recognized as FormAction at all — no FormAction was detected even though a standard `<form>` element is present in the DOM. The component tree structure produced by Angular Material prevents the `CandidateActionExtractor` from matching the `<form>` element's expected attributes.

Finality mechanism and zero-MongoDB outcomes

AIRBUS demonstrated that the `FINALITY_SYSTEM_PROMPT` can return `[False, False, ..., False]` for all features at a state, resulting in zero MongoDB writes even when feature extraction completes. This is correct behavior — field clicks do not conclude any feature without form submission. However, it means that for fully auth-blocked apps with standard forms, the pipeline produces no MongoDB artifacts at all. Downstream systems that depend on MongoDB records will receive nothing for these apps.

LLM inference risk

Feature descriptions are LLM inferences from screenshots and HTML. HEROES produced 58 raw features, many of which are near-duplicate semantic variants of "delete character from collection" (one per hero delete button). After vector deduplication, ~25 remain — but these are still inferences, not verified by actual CRUD execution. POSTUSERS produced features like "send user invitation" that are not implemented in the application. SOFTSCANNER_CQA produced 5 near-identical "navigate to next step in form wizard" records (one per ISO quality characteristic toggle button) — all stored as separate MongoDB documents because vector deduplication via Atlas `$vectorSearch` inserts first and deduplicates on subsequent similar features only. These cases illustrate that feature hallucination and duplication risk increases when the LLM encounters repetitive UI patterns (multiple buttons with the same icon and similar semantic context).

Data state mutation risk

JSON Server's `db.json` is mutated by PUT/DELETE requests during the run. The pipeline's critical-action detection correctly skipped delete operations, but edit operations (which update records) may have been executed if not detected as critical. Subsequent state visits would then see modified records. Without a reset between action executions (json-server-reset middleware is included but not triggered by the pipeline), the db.json state is non-deterministic across runs.

Authentication wall

EVERTRADUORA was an effectively partial run (3 public pages out of 10+). AIRBUS was the most restricted run — only 1 public page, 0 MongoDB records. These runs demonstrate that the pipeline cannot evaluate applications where authentication blocks all or most routes.

Empty backend state

POSTUSERS uses in-memory storage starting empty. `/users/:id` was structurally unreachable.

External database dependency

AIRBUS introduced a new setup dimension: an external MySQL database that is not managed by any Docker Compose file in the repository. The database state is ephemeral (Docker container started and removed per run); without persistent MySQL storage, the application cannot be started without repeating the setup steps. This makes AIRBUS the most operationally fragile benchmark to reproduce.

Inline/modal form states

HEROES demonstrated that inline form states (same URL, different DOM) are correctly captured as distinct state hashes. However, the pipeline replays actions from the beginning to re-reach these states (`crawl_context.load_state` replays the crawl path). For inline forms opened by clicking a button within a list, this replay depends on the list still showing the same elements — a dependency on backend data consistency.

Single environment

All runs on Windows 11, Python 3.11, Chrome, Node.js 22. The `NODE_OPTIONS=--openssl-legacy-provider` fix for HEROES and AIRBUS is Node.js-version-specific. On Node.js 16, it would not be needed. The fix applies to Angular ≤12 on Node.js ≥17.

Angular Material SPA action explosion

SOFTSCANNER_CQA has 452 candidate actions across 9 states — far more per-state than any other benchmark (PETCLINIC: ~8/state, HEROES: ~10/state). Angular Material components generate deeply nested DOM trees with many clickable sub-elements per logical UI widget. The 30-action ceiling is consumed rapidly on initial form field interactions (mat-form-field wrappers, inner inputs) before reaching higher-level workflow controls. This is a distinct failure mode from auth walls or empty backends: the pipeline reaches the application but is unable to advance through the primary workflow because widget interactions dominate the action budget.

External apps not repository-controlled

POSTUSERS, HEROES, AIRBUS, and SOFTSCANNER_CQA are not under the AutoE2E repository. Changes to these apps between runs affect results without tracking.

35. Final Conclusion

What AutoE2E actually is

AutoE2E is a **feature discovery and action-feature mapping engine**. It crawls web applications using Selenium, uses LLMs to understand page context from screenshots and DOM, extracts testable feature descriptions, deduplicates them via vector embeddings, and stores action-to-feature mappings in MongoDB.

What AutoE2E is not

AutoE2E is **not a test generator**. It does not produce executable test cases, test scripts, assertions, or test files in any format. This conclusion is supported by static code inspection and confirmed across six independent benchmark runs.

What was empirically demonstrated across six runs

PETCLINIC (2026-03-10): 9 states, 72 actions, 46 features stored, ~14 min. Broadest state/URL spread (9 states, 7 URLs) and highest feature storage (46 records) of all six runs. Forms submittable only because of PetClinic's custom `data-testid` instrumentation. Binding constraint: 30-action ceiling.

EVERTRADUORA (2026-03-12): 3 states, 13 actions, 5 features stored, ~10 min. Authentication wall blocked all core functionality. Exposed 4 pipeline defects. Binding constraint: auth wall.

POSTUSERS (2026-03-12): 7 states, 95 total actions (30 executed), ~37 features stored, ~10 min. External app, config-only integration. Empty backend limited depth. Richest total action set (95) but all list pages empty. Binding constraint: 30-action ceiling.

HEROES (2026-03-12): 5 states, 53 total actions (30 executed), 58 raw features (~25 stored after deduplication), ~10 min. External app, config-only integration. JSON Server seed data enabled populated list pages and inline form state discovery. Run behavior was consistent with critical-action filtering on delete buttons (none executed). No new pipeline defects. Binding constraint: 30-action ceiling.

AIRBUS (2026-03-12): 1 state, 3 candidate actions, 6 action-processing iterations across 2 revisits, 4 raw LLM features, **0 features stored**, ~5 min. External app, config-only integration. Full JWT authentication blocked all routes except the login page. FormAction was detected but could not be submitted. Finality check returned `False` for all 4 raw features — first run with zero MongoDB artifacts. Binding constraint: auth wall + no action reaching a finalized feature state.

SOFTSCANNER_CQA (2026-03-12): 9 states, 9 transitions, 452 total candidate actions (30 executed), 42 raw LLM features, **6 features stored**, ~13 min. External app, config-only integration. Cleanest setup of all six runs (Angular 17 esbuild, no Docker, no auth, pre-built backend). All 9 states at a single URL — distinct DOM configurations from Angular Material tree toggles and dropdown opens. FormAction not detected despite a standard `<form>` element (Angular Material component tree not recognized by `CandidateActionExtractor`). The "Start Assessment" primary workflow button was not reached within the ceiling. Binding constraint: 30-action ceiling consumed by Angular Material widget interactions.

Generalizability assessment across six runs

Capability	PETCLINIC	EVERTRADUORA	POSTUSERS	HEROES	AIRBUS	SOFTSCANNER_CQA
State discovery	✓	✓ (limited)	✓	✓	✓ (1 state only)	✓ (9 DOM states, 1 URL)
LLM context extraction	✓	✓	✓	✓	✓	✓
Feature extraction (raw)	✓	✓ (minimal)	✓	✓	✓ (4 raw)	✓ (42 raw)
Feature storage (MongoDB)	✓	✓	✓	✓	**✗ (0 records)**	✓ (6 final of 42)
Form submission	✓ (custom attrs)	✗	✗	N/A	✗	✗
FormAction detection	✓	✓	✓	N/A (not detected)	✓	**✗ (Angular Material not recognized)**
Auth handling	N/A	✗	N/A	N/A	✗	N/A
External app integration	N/A	N/A	✓	✓	✓	✓
Inline DOM state capture	N/A	N/A	N/A	✓	N/A	✓ (9 DOM states, 1 URL)
Critical action detection	Not tested	Not tested	Not tested	Consistent with filtering	Not tested	Not tested
External DB dependency	Docker (managed)	Docker Compose (managed)	None	None	**Manual (Docker MySQL)**	None

Does the title overstate the implementation?

Yes, more strongly after six benchmark runs. PETCLINIC was the designed benchmark with custom HTML instrumentation. EVERTRADUORA, POSTUSERS, HEROES, AIRBUS, and SOFTSCANNER_CQA — each representing more typical web application patterns — demonstrate that the pipeline cannot submit forms in standard applications, cannot handle authentication, and for applications with Angular Material component trees, cannot even detect forms as FormAction. AIRBUS is the clearest example of zero-output: the pipeline found the login form, extracted conceptual features, and produced nothing storable — because no interaction with a standard login form concludes any feature without an authenticated session. SOFTSCANNER_CQA demonstrates a complementary failure mode: the application is fully accessible, but the 30-action ceiling is exhausted on Angular Material widget interactions before the primary workflow (quality assessment submission) can be initiated. The title "Feature-Driven End-To-End Test Generation" overstates both the output type (intermediate data, not tests) and the generality of the current implementation.

Why the outputs remain useful (within documented scope)

The six runs together produce: 34 state graph nodes across 30+ distinct application states, 192+ raw feature descriptions (46+5+37+58+4+42), 34 screenshots, and comprehensive action-element records across six application types. These artifacts constitute a varied dataset of navigation maps, LLM-inferred feature inventories, and form structure documentation — useful intermediate data for any downstream system attempting to generate executable tests from discovered application behavior. AIRBUS remains the minimum observed artifact set for fully auth-blocked applications; SOFTSCANNER_CQA demonstrates that high-action-density Angular Material SPAs require a substantially higher action ceiling to reach primary workflow entry points.